EXPERIMENT 5

Write the Python program for the Missionaries and Cannibals problem

Register Number: **192511013**

192511013
192511013
192511013

Aim

To write a Python program that moves 3 missionaries and 3 cannibals from the left bank to the right bank of a river using a boat that carries at most 2 people, such that the cannibals never outnumber the missionaries on either bank (where missionaries are present), using Breadth First Search.

Algorithm

Step 1: Represent a state as (m, c, boat), where m and c are the numbers of missionaries and cannibals on the left bank and boat = 1 if the boat is on the left bank, 0 if on the right. Start = (3, 3, 1), Goal = (0, 0, 0).

Step 2: A state is valid if 0 <= m <= 3, 0 <= c <= 3 and, on each bank, either there are no missionaries or missionaries >= cannibals.

Step 3: Put the start state in a queue and keep a parent table for path reconstruction.

Step 4: Remove the front state from the queue. If it is the goal, trace the parent links back to the start and stop.

Step 5: Otherwise generate every possible boat load (1 or 2 people: 1M, 2M, 1C, 2C, 1M+1C) and move it in the direction opposite to the boat's current bank.

Step 6: Keep only the valid, unvisited new states, record their parent and the move, and add them to the queue. Repeat from step 4.

Step 7: Print each state of the shortest solution path with the move made, and the total number of crossings.

192511013

Program

```
from collections import deque

M = C = 3          # missionaries and cannibals on the left bank at the start
BOAT = 2           # boat capacity

def valid(m, c):
    if not (0 <= m <= M and 0 <= c <= C):
        return False
    if m > 0 and c > m:                       # left bank unsafe
        return False
    if (M - m) > 0 and (C - c) > (M - m):     # right bank unsafe
        return False
    return True

def successors(state):
    m, c, boat = state                        # boat = 1 -> left bank, 0 -> right
    sign = -1 if boat == 1 else 1
    for dm in range(BOAT + 1):
        for dc in range(BOAT + 1 - dm):
            if dm + dc == 0:
                continue
            nm, nc = m + sign * dm, c + sign * dc
            if valid(nm, nc):
                yield (nm, nc, 1 - boat), (dm, dc)

def bfs():
    start, goal = (M, C, 1), (0, 0, 0)
    queue = deque([start])
    parent = {start: (None, None)}
    while queue:
        state = queue.popleft()
        if state == goal:
            path = []
            while state is not None:
                prev, move = parent[state]
                path.append((state, move))
                state = prev
            return path[::-1]
        for nxt, move in successors(state):
            if nxt not in parent:
                parent[nxt] = (state, move)
                queue.append(nxt)
    return None

if __name__ == "__main__":
    path = bfs()
    print(f"{'Step':<5}{'Left (M,C)':<13}{'Right (M,C)':<13}{'Boat':<7}Move")
    print("-" * 62)
    for i, ((m, c, b), move) in enumerate(path):
        side = "Left" if b else "Right"
        mv = "Start" if move is None else f"{move[0]}M {move[1]}C cross " + \
             ("L->R" if b == 0 else "R->L")
        print(f"{i:<5}{str((m, c)):<13}{str((M - m, C - c)):<13}{side:<7}{mv}")
    print("\nTotal crossings:", len(path) - 1)
```

Output

```
Step Left (M,C)   Right (M,C)  Boat   Move
-------------------------------------------------------------
0    (3, 3)       (0, 0)       Left   Start
1    (3, 1)       (0, 2)       Right  0M 2C cross L->R
2    (3, 2)       (0, 1)       Left   0M 1C cross R->L
3    (3, 0)       (0, 3)       Right  0M 2C cross L->R
4    (3, 1)       (0, 2)       Left   0M 1C cross R->L
5    (1, 1)       (2, 2)       Right  2M 0C cross L->R
6    (2, 2)       (1, 1)       Left   1M 1C cross R->L
7    (0, 2)       (3, 1)       Right  2M 0C cross L->R
8    (0, 3)       (3, 0)       Left   0M 1C cross R->L
9    (0, 1)       (3, 2)       Right  0M 2C cross L->R
10   (0, 2)       (3, 1)       Left   0M 1C cross R->L
11   (0, 0)       (3, 3)       Right  0M 2C cross L->R

Total crossings: 11
```

192511013